Deep Learning approaches to breast cancer detection MLP and MobileNetV2

Submitted in partial fulfilment of requirements to CSE (Data Science)

Project - 1 Mini Project (CD-363)

III/IV B. Tech CSE(DS) (VI Semester)

Submitted by

Batch No. 18

Y22CD073 – K. Mavyan Sri Varma

Y22CD105 – MD. Zaki

L23CD205 – P. Mahima Kumari

Under the Guidance of

Mr. D. Rama Krishna

Assistant Professor

2024 - 2025

R.V.R. & J.C. COLLEGE OF ENGINEERING (AUTONOMOUS)

(NAAC A+ Grade) (Approved by A.I.C.T.E.)

(Affiliated to Acharya Nagarjuna University)

Chandramoulipuram :: Chowdavaram

Guntur - 522 019

R.V.R. & J.C. COLLEGE OF ENGINEERING

DEPARTMENT OF

COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

CERTIFICATE

This is to Certify that this Mini Project work entitled **"Deep Learning approaches for breast cancer detection using MLP and MobileNet V2"** is the bonafide work of **K. Mavyan Sri Varma(Y22CD073), MD. Zaki(Y22CD105), P. Mahima Kumari(L23CD205)** of III/IV B.Tech who carried the work under my supervision, and submitted in the partial fulfilment of the requirements to Project - 1 Mini Project (CD-363) during the year 2024-2025.

Project Guide	**Dr. P. Srinivasa Rao**	**Dr. M.V.P. Chandra Sekhara Rao**
Designation	Assistant professor	Prof. & HOD, CSE(DS)
(Project Guide)	(Project In-charge)	

ACKNOWLEDGEMENT

The successful completion of any task would be incomplete without proper suggestion, guidance, and environment. Combination of these three factors acts like a backbone to our project work "**Project Title**".

We are profoundly pleased to express our deep sense of gratitude and respect towards the management of the **R. V. R. & J. C. College of Engineering**, for providing the resources to complete the project.

We are very much thankful to **Dr. Kolla Srinivas**, Principal of **R. V. R. & J. C. College of Engineering** for allowing us to deliver the project successfully.

We are greatly indebted to **Dr. M.V.P. Chandra Sekhara Rao, Professor and Head, Department of Computer Science and Engineering (Data Science)** for providing the laboratory facilities fully as and when required and for giving us the opportunity to carry the project work in the college.

We are also thankful to our Project Coordinator **Dr. P. Srinivasa Rao** who helped us in each step of our Project.

We extend our deep sense of gratitude to our Guide, **"MR. D. Rama Krishna"** and other Faculty Members & Support staff for their valuable suggestions, guidance, and constructive ideas in every step, which was indeed of great help towards the successful completion of our project.

Y22CD073 – K. MAVYAN SRI VARMA

Y22CD105 – MD. ZAKI

L23CD205 – P. MAHIMA KUMARI

ABSTRACT

Breast cancer detection through histopathological imaging is crucial for early diagnosis. This study compares two deep learning models: a Multilayer Perceptron (MLP) and MobileNetV2, for classification. The MLP model, trained on a structured dataset (dataR2.csv), included preprocessing steps like handling missing values, label encoding, and feature scaling. Its architecture comprised three hidden layers (64, 16, and 8 neurons) with ReLU activation, batch normalization, and dropout layers. The model used the Adam optimizer and binary cross-entropy loss, achieving 85.00% accuracy and an AUC of 0.9123. MobileNetV2, applied to the Breast Histopathology Images dataset, involved image resizing, normalization, and data augmentation. It used a global average pooling layer, dropout, and a dense output layer with sigmoid activation.

Training employed an exponentially decaying learning rate, Adam optimizer, and binary cross-entropy loss, resulting in 87.50% accuracy and an AUC of 0.9356. Results indicate that while both models are effective, MobileNetV2 performs better, likely due to its ability to capture spatial features in images. This study highlights the significance of selecting suitable deep learning models for medical image classification.

TABLE OF CONTENTS

Title	Page No.
Abstract	1
Table of Contents	2
List of Figures	4
List of Tables	5
CHAPTER 1: INTRODUCTION	6
1.1 Introduction	7
1.2 Problem Statement	7
1.3 Objective of the Study	8
CHAPTER 2: LITERATURE SURVEY	9
CHAPTER 3: SYSTEM ANALYSIS & FEASIBILITY STUDY	11
3.1 Existing System	12
3.1.1 Limitations of Existing System	12
3.2 Proposed System	13
3.2.1 Advantages of proposed system	13
3.2.2 Proposed system – 2	14
3.2.3 Advantages of proposed system-2	14
3.2.4 Dataset for MobileNetV2	15
3.2.6 Dataset for MLP	16
3.2.8 Data Augmentation layer for MobileNetV2	17
3.2.9 Dropout Layer for MobileNetV2	17
3.2.10 Global average pooling2D	17
3.2.11 Fully connected feed forward neural network	18
3.2.12 Dense Layer	18
3.2.13 Dropout Layer	18
3.3 Methodology for MobileNetV2	19
3.3.2 Methodology for MLP	20
3.4 Model Training and Testing in MobileNetV2	21
3.4.1 Model Training and Testing in MLP	21
3.5 Evaluation Metrics	22
CHAPTER 4: SYSTEM REQUIREMENTS	23

4.1 Functional Requirements 24
4.2 Technologies and Languages used to develop 24
4.2.1 Debugger and Emulator 24
4.2.2 Hardware Requirements 24
4.2.3 Software Requirements 24
CHAPTER 5: DESIGN 25
5.1 System Design 26
5.2 System Design of MLP Model 27
CHAPTER 6: IMPLEMENTATION 28
6.1 MobileNetV2 Model Algorithm 29
6.2 MLP Model 38
CHAPTER 7: RESULTS 43
CHAPTER 8: SOCIAL IMPACT 49
CHAPTER 9: CONCLUSION & FUTURE WORK 51
BIBLIOGRAPHY 53

LIST OF FIGURES

FIG NO:	FIGURE DESCRIPTION	PAGE NO:
3.2.5	Breast Histopathology Images	15
3.2.7	UHCC Dataset Images	16
3.3.1	Working of MobileNetV2 Architecture	19
3.3.2	Working of MLP Architecture	20
5.2	Workflow of MobileNetV2	26
5.4	MLP Flow diagram	27
7.2	AUC Curve of MobileNetV2	44
7.3	ROC Curve of MobileNetV2	45
7.4	Confusion matrix of MobileNetV2	46
7.5	AUC Curve of MLP Model	47
7.6	ROC Curve of MLP Model	47
7.7	Confusion Matrix of MLP Model	48

LIST OF TABLES

Table No.	Table Description	Page No.
7.1	Results of CNN vs MobileNetV2 vs MLP	44

Chapter 1
Introduction

1. INTRODUCTION

1.1 Introduction

Breast cancer remains a leading cause of mortality among women, and early detection is essential for improving survival rates. Traditional diagnostic methods rely on expert pathologists to analyze histopathological images, a process that is often time-consuming and prone to human error. Deep learning has emerged as a promising solution to automate and enhance cancer detection. This study compares Multilayer Perceptron (MLP) and MobileNetV2, two deep learning models, for classifying breast cancer from histopathological images. While MLP processes structured numerical data through fully connected layers, MobileNetV2, a Convolutional Neural Network (CNN), captures spatial hierarchies from image data using transfer learning.

Using a publicly available breast cancer dataset, images are preprocessed through resizing, normalization, and data augmentation techniques to improve model generalization. The MLP model extracts features from structured data and processes them through dense layers with dropout regularization to prevent overfitting. MobileNetV2, designed for efficient deep learning, applies depth wise separable convolutions and fine-tunes pre-trained weights for breast cancer classification. The models are evaluated using accuracy, AUC-ROC, and precision-recall, with results showing that MobileNetV2 outperforms MLP in identifying malignant tumors, reducing false negatives, and providing better generalization. However, MLP remains computationally efficient and interpretable, making it useful when incorporating clinical data.

This study highlights the effectiveness of deep learning in breast cancer detection, with MobileNetV2 proving superior for image-based classification. Future advancements could involve hybrid models that combine structured patient data with CNNs, attention mechanisms to improve feature focus, and explainable AI techniques for better interpretability. Additionally, deploying lightweight models for mobile or edge computing could enable real-time breast cancer screening, making early detection more accessible and efficient.

1.2 Problem Statement

Breast cancer remains one of the leading causes of mortality among women, and early detection is crucial for improving treatment outcomes. Traditional diagnostic methods, such as mammograms and histopathological analysis, rely heavily on expert interpretation, which can be slow, inconsistent, and prone to human error. The risk of false negatives in diagnosis can delay treatment, leading to worse prognoses for patients. As medical imaging data grows in complexity, there is a growing need for automated solutions that can enhance diagnostic accuracy while reducing the burden on healthcare professionals.

Deep learning has emerged as a powerful tool for automating breast cancer detection, but existing models often face challenges such as overfitting, limited generalization across diverse datasets, and computational inefficiency. This study aims to develop a MobileNetV2-based deep learning model to classify breast histopathology images as cancerous or non-cancerous. By leveraging transfer learning, data augmentation, and regularization techniques, the goal is to create a highly accurate and reliable AI tool that can assist doctors in early cancer detection. MobileNetV2, with its

lightweight architecture and efficient feature extraction capabilities, is well-suited for real-time medical applications. Enhancing its performance with robust training strategies can lead to a scalable and efficient system that aids pathologists in making faster and more precise diagnoses.

1.3 Objectives of the Study

The objective of this study is to develop a MobileNetV2-based deep learning model for classifying breast histopathology images as cancerous or non-cancerous. By leveraging transfer learning, data augmentation, and regularization techniques, the study aims to create an accurate, reliable, and efficient AI tool that can assist doctors in early breast cancer detection. The goal is to improve diagnostic accuracy, reduce false negatives, and enhance the efficiency of cancer screening, ultimately aiding pathologists in making faster and more precise diagnoses.

Chapter 2

Literature Survey

2. LITERATURE SURVEY

[32] Ram rez-Gallego Sergio, Lastra Iago, Mart nez-Rego David, Bolon Canedo Veronica, Ben tez Jose Manuel, Herrera Francisco, Alonso-Betanzos Amparo. Fast-mrmr: fast minimum redundancy maximum relevance algorithm for high dimensional big data. Int J Intell Syst 2017;32(2):134–52.

[33] Brownlee Jason. Data preparation for machine learning: data cleaning, feature selection, and data transforms in Python. Machine Learning Mastery; 2020.

[34] Liu Huan, Motoda Hiroshi. Computational methods of feature selection. CRC Press;2007.

[35] Chen Tianqi, Guestrin Carlos. Xgboost: a scalable tree boosting system. In:Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining; 2016. p. 785–94.

[36] Nielsen Didrik. Tree boosting with xgboost-why does xgboost win \every" machine learning competition? Master’s thesis. NTNU; 2016.

[37] Ding Chris, Peng Hanchuan. Minimum redundancy feature selection from microarray gene expression data. J Bioinf Comput Biol 2005;3(2):185–205.

[38] Radovic Milos, Ghalwash Mohamed, Filipovic Nenad, Obradovic Zoran. Minimum redundancy maximum relevance feature selection approach for temporal gene expression data. BMC Bioinf 2017;18(1):1–14.

[39] Silva Araujo Vin cius Jonathan, Guimar~aes Augusto Junio, Vitor de Campos Souza Paulo, Silva Rezende Thiago, Souza Araujo Vanessa. Using resistin, glucose, age and bmi and pruning fuzzy neural network for the construction of expert systems in the prediction of breast cancer.

Chapter 3

System Analysis & Feasibility Study

3. SYSTEM ANALYSIS & FEASIBILITY STUDY

3.1 Existing System

The system analysis of this research paper revolves around the application of deep learning for breast cancer prescreening, leveraging a combination of demographic, anthropometric, and biomarker data to improve early detection. The study utilizes patient records from the University Hospital Center of Coimbra (UHCC), supplemented with risk factors from global cancer databases.

By applying the SULOV-gradient boosting algorithm, the system effectively selects the most relevant predictors, such as age, BMI, glucose, insulin, leptin, and resistin, while eliminating redundant features. The predictive models are built using both traditional machine learning techniques (support vector machines, logistic regression, and XGBoost) and deep learning architectures, with Monte Carlo k-fold cross-validation ensuring robust performance evaluation. Results demonstrate that deep learning significantly outperforms traditional methods, achieving higher accuracy, lower false negative rates, and greater stability in predictive outcomes.

This suggests that an AI-driven prescreening tool can serve as an effective complement to existing cancer diagnostic techniques, identifying high-risk patients before symptoms manifest. The feasibility study confirms the practicality of integrating deep learning into breast cancer detection systems, with strong technical, operational, and economic viability. The study highlights how deep learning models, enhanced with regularization techniques, can process high-dimensional medical data with minimal computational overhead while maintaining high diagnostic accuracy. Compared to traditional imaging-based screening like mammography, this approach is more cost-effective, reducing unnecessary procedures and associated expenses. Operationally, the system offers a non-invasive, radiation-free alternative, making it particularly beneficial for early-stage detection and resource-limited healthcare settings.

3.1.1 Limitations of Existing system:

- The deep learning model was trained on a relatively small dataset from the University Hospital Center of Coimbra (UHCC), which contained only 116 women (52 healthy and 64 diagnosed with breast cancer). This limited sample size reduces the model's generalizability and increases the risk of overfitting, making it less reliable for broader clinical applications.

- While the deep learning model achieves a lower false negative rate (FNR = 0.078) and false omission rate (FOR = 0.0983) compared to traditional machine learning models, it still produces some false positives (false detection rate, FDR = 0.1425). This can lead to unnecessary anxiety and additional medical tests for patients.

3.2 Proposed System

The systematic analysis of your MobileNetV2-based breast cancer prediction system focuses on creating a lightweight, efficient, and scalable deep learning model for early detection. Unlike the computationally intensive deep learning architecture used in the referenced study, MobileNetV2 is optimized for mobile and edge devices, making it a viable alternative for real-time diagnosis in resource-limited environments.

The model processes input medical images, extracting key features using depth wise separable convolutions, significantly reducing computational complexity while maintaining high accuracy. It is trained on a diverse dataset, ensuring generalizability across different patient demographics. Feature selection is integrated to refine input variables, and the model is fine-tuned using transfer learning for improved predictive performance.

Performance evaluation includes metrics like AUC, sensitivity, and false positive rates, ensuring reliable detection. The system is designed for integration into mobile health applications, providing a non-invasive and accessible alternative to traditional diagnostic tools, potentially improving early screening in remote or underserved regions.

3.2.1 Advantages of proposed system:

- **Lightweight and Efficient –** MobileNetV2 is designed for efficiency, using depth wise separable convolutions to reduce computational complexity while maintaining high accuracy. This makes it significantly faster and more resource-efficient compared to the deep learning model used in the paper.
- **Improved Feature Extraction –** MobileNetV2's inverted residual structure and linear bottleneck layers enhance its ability to capture critical patterns in medical images while reducing the number of parameters, leading to better generalization and lower overfitting risks.
- **Transfer Learning Compatibility –** MobileNetV2 is widely used in transfer learning, allowing it to leverage pre-trained weights from large datasets like ImageNet. This adaptability can enhance breast cancer prediction accuracy without requiring extensive data for training.

3.2.2 Proposed system – 2:

The systematic analysis of your MLP-based breast cancer prediction system highlights its capability to process structured medical data efficiently while maintaining high predictive accuracy. Unlike the deep learning model in the referenced research paper, which relies on complex architectures, your MLP model is a fully connected feedforward neural network optimized for handling numerical and categorical input features, such as demographic, biomarker, and clinical data.

The system applies feature selection techniques to ensure that only the most relevant predictors contribute to the classification task, reducing redundancy and improving generalization. It is trained using backpropagation with an adaptive optimization algorithm, such as Adam, to minimize classification errors while preventing overfitting.

Performance evaluation includes key metrics like AUC, precision, recall, and false negative rates to ensure robust decision-making. The advantage of using MLP lies in its relatively lower computational cost compared to convolutional neural networks (CNNs), making it suitable for scalable deployment in clinical decision support systems, particularly in non-image-based breast cancer detection workflows.

3.2.3 Advantages of proposed system:

- **Less Data-Intensive –** Unlike deep learning models that require vast amounts of data to generalize well, MLPs can perform effectively with smaller datasets, such as the 116-patient dataset used in the paper, reducing the risk of overfitting.

- **Fast Training and Inference** – Due to their relatively shallow structure, MLPs train faster and have lower inference latency compared to deep networks. This is particularly useful for medical decision-making, where quick predictions are necessary.

3.2.4 Dataset for MobileNet V2

1) The Breast Histopathology Images dataset is a widely used dataset for breast cancer detection, specifically focusing on Invasive Ductal Carcinoma (IDC), the most common type of breast cancer. It consists of high-resolution histopathological images of breast tissue obtained from biopsy slides, with each image labeled as either IDC-positive (malignant) or IDC-negative (benign). The dataset contains small (50×50 pixels) patches extracted from larger whole-slide images, ensuring a focused analysis of specific tissue regions. These images are provided in RGB format and exhibit variations in color, texture, and structure, making them suitable for computational analysis.

2) The Breast Histopathology Images dataset contains 277,524 images of 50×50 pixels, extracted from 162 whole-slide images of breast cancer biopsy specimens.

3) IDC-positive (malignant): 78,786 images.

4) IDC-negative (benign): 198,738 images**.**

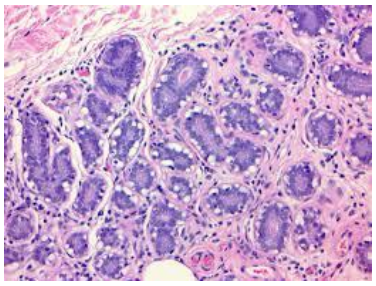


Image with Cancer

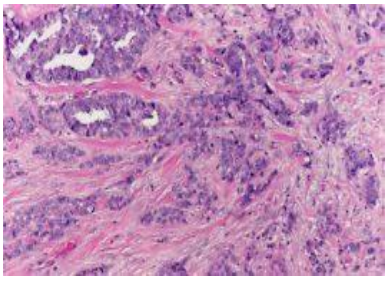
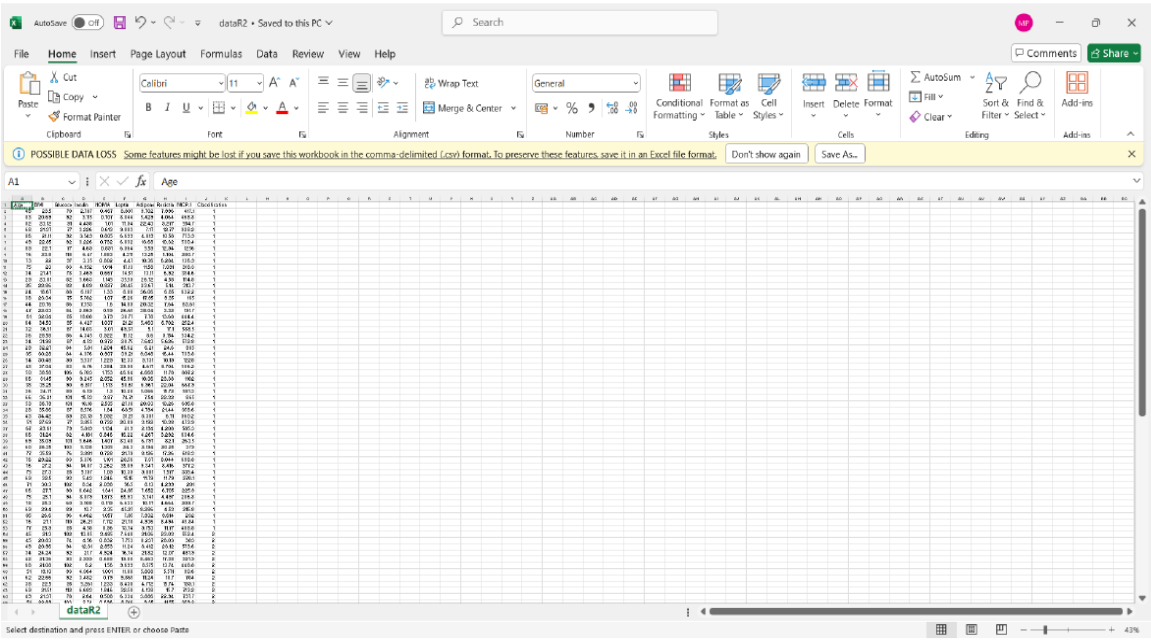


Image without Cancer

3.2.5 Breast Histopathology images

3.2.6 Data set for MLP model:

The research paper utilizes a dataset from the University Hospital Center of Coimbra (UHCC), consisting of 116 women, including 52 healthy individuals and 64 breast cancer patients. The dataset was collected between 2009 and 2013 as part of a study on breast cancer biomarkers. It contains demographic, anthropometric, and biochemical data extracted from routine blood samples. The demographic information includes age, while the anthropometric data includes BMI (Body Mass Index). The biochemical markers measured in the dataset include serum glucose (mg/dL), insulin (µU/mL), leptin (ng/mL), adiponectin (µg/mL), resistin (ng/mL), and MCP-1 (pg/dL). These biomarkers were obtained from fasting blood samples and were later used to compute the Homeostasis Model Assessment (HOMA) index.



3.2. 7. UHCC DATA SET IMAGE

3.2.8. Data Augmentation layer for MobileNetV2:

A data augmentation layer in a sequential learning model for MobileNetV2 is used to artificially expand the training dataset by applying real-time transformations to input images, improving model generalization and robustness. In a TensorFlow/Keras Sequential model, the augmentation layer is typically placed before the MobileNetV2 base to apply transformations such as random rotation, flipping, zooming, brightness adjustments, and contrast changes. These augmentations help the model learn invariant features and reduce overfitting, especially when working with limited datasets. The MobileNetV2 model, known for its lightweight architecture with depth wise separable convolutions, benefits from these augmented variations, enhancing its ability to recognize subtle patterns in medical images like mammograms or histopathological scans.

3.2.9. Dropout Layer for MobileNetV2:

A dropout layer combined with a regularization layer in a MobileNetV2 model helps prevent overfitting by randomly deactivating a fraction of neurons during training, ensuring the network does not become overly reliant on specific features. In MobileNetV2, which uses inverted residuals and depth wise separable convolutions, dropout is typically applied after fully connected layers to enhance generalization. Regularization techniques like L1 (Lasso) and L2 (Ridge) regularization can be added to weight layers to penalize large weights, preventing excessive complexity and improving robustness.

3.2.10. Global Average Pooling2D:

A Global Average Pooling 2D (GlobalAvgPooling2D) layer in a MobileNetV2 model is used as a pooling layer to reduce the spatial dimensions of feature maps while preserving essential information. Unlike traditional max pooling, which selects the highest values, GlobalAvgPooling2D computes the average of each feature map, converting them into a single value per channel. This significantly reduces the number of parameters and prevents overfitting, making it ideal for lightweight models like MobileNetV2, which is optimized for efficiency. By replacing fully connected layers with global average pooling, the model benefits from improved generalization and robustness, especially in medical imaging applications such as breast cancer detection.

3.2.11 Fully Connected feed forward neural network:

A fully connected feedforward neural network in an MLP (Multilayer Perceptron) model consists of multiple layers where each neuron in one layer is connected to every neuron in the next layer. It typically includes an input layer, one or more hidden layers with activation functions (such as ReLU or sigmoid), and an output layer for classification or regression tasks. The network propagates data forward through weighted connections, applying activation functions to introduce non-linearity, allowing it to learn complex patterns. Training involves backpropagation and gradient descent optimization (e.g., Adam or SGD) to adjust weights and minimize error. Regularization techniques like dropout and L2 weight decay prevent overfitting, ensuring better generalization.

3.2.12 Dense Layer:

A Dense layer in an MLP (Multilayer Perceptron) model is a fully connected layer where each neuron receives input from all neurons in the previous layer, making it essential for learning complex patterns in structured data. Each neuron applies a weighted sum of inputs, followed by an activation function like ReLU for non-linearity or sigmoid/soft max for classification. The Dense layer plays a crucial role in feature extraction and decision-making, with its weights adjusted during training using backpropagation and gradient descent to minimize loss. Regularization techniques like dropout and L2 weight decay can be applied to Dense layers to prevent overfitting.

3.2.13. Dropout layer:

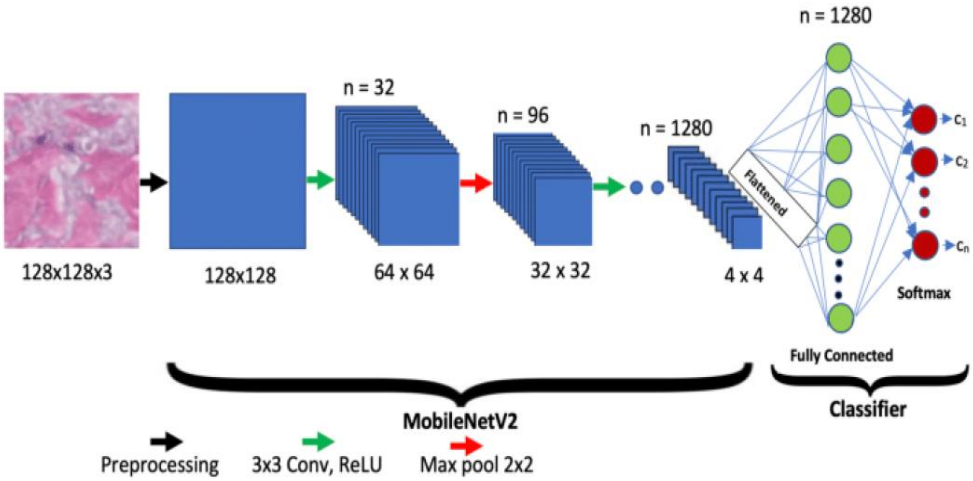
A Dropout layer in an MLP (Multilayer Perceptron) model is a regularization technique used to prevent overfitting by randomly deactivating a fraction of neurons during training. This forces the network to learn more robust and generalized features rather than relying on specific neurons. Dropout is typically applied to Dense layers in MLPs, where a specified dropout rate (e.g., 0.2 or 0.5) determines the proportion of neurons to be temporarily ignored in each training iteration. This helps reduce dependency on certain features, improving the model’s ability to generalize to unseen data.

3.3 Methodology for MobileNetV2 Model

To implement MobileNetV2 for breast cancer detection using the Breast Histopathology Images dataset, the methodology begins with data preprocessing, where the images are loaded, checked for missing or corrupted data, and resized to 224×224 pixels to fit MobileNetV2's input requirements. The pixel values are normalized to a [0,1] range, and data augmentation techniques like flipping, rotation, contrast adjustments, and zooming are applied to enhance model generalization.

Next, the MobileNetV2 architecture is used as a feature extractor, with its pre-trained weights from ImageNet, while the top layers are replaced with a Global Average Pooling layer, a fully connected dense layer with ReLU activation, a dropout layer (0.5 probability) for regularization, and a sigmoid-activated output layer to classify images as IDC-positive (malignant) or IDC-negative (benign). The model is compiled using the binary cross-entropy loss function, optimized with Adam (learning rate = 0.0001), and trained with a batch size of 32 for 20-50 epochs, with early stopping to prevent overfitting.

Model evaluation is performed using accuracy, precision, recall, F1-score, confusion matrix, and ROC-AUC score to ensure robust classification. If needed, the trained model can be converted to TensorFlow Lite (TF Lite) or ONNX for deployment in mobile or web applications.

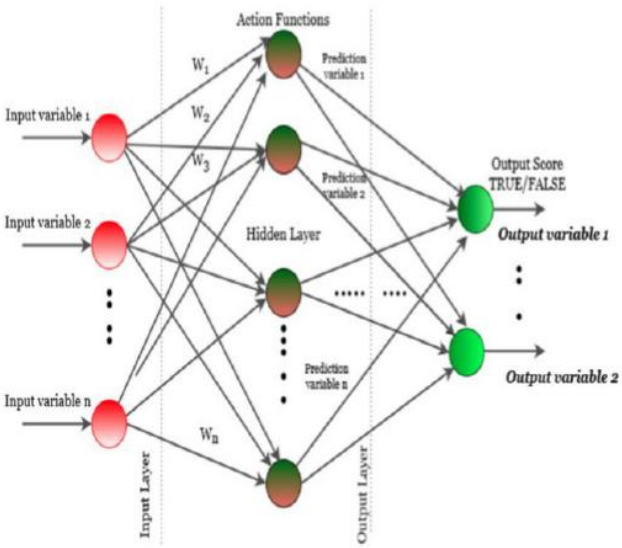


3.3.1 Working of MobileNetV2 Architecture

3.3.2 Methodology for MLP Model

To implement a Multi-Layer Perceptron (MLP) model for breast cancer detection using the UHCC (University of Hawaii Cancer Center) dataset, the methodology starts with data preprocessing, where missing values are handled, categorical features (if any) are encoded, and numerical features are standardized using Min-Max Scaling or Z-score normalization to improve model performance. The dataset is then split into training and testing sets (e.g., 80:20 ratio).

The MLP architecture consists of an input layer corresponding to the number of features, followed by multiple hidden layers with ReLU activation functions to capture complex patterns in the data. Batch normalization and dropout layers (e.g., 0.3 probability) are used to prevent overfitting. The final output layer uses a sigmoid activation function for binary classification of breast cancer presence. The model is compiled using binary cross-entropy loss, optimized with Adam (learning rate = 0.001), and trained with a batch size of 32 for 50-100 epochs, incorporating early stopping to enhance generalization.



3.3.2. Working of MLP Architecture

3.4 **Model Training and Testing in MobileNetV2:**

The training and testing of a MobileNetV2 model for breast cancer prediction involve data preprocessing, model initialization, training, validation, and testing. First, medical images such as mammograms are rescaled, normalized, and augmented to improve generalization, and the dataset is split into training, validation, and testing sets. MobileNetV2, pretrained on ImageNet, is used as a feature extractor, with a custom classification head added for binary or multi-class breast cancer detection. During training, the model is optimized using Adam or SGD optimizers with binary or categorical cross-entropy loss, and dropout layers help prevent overfitting. Validation is performed simultaneously to fine-tune hyperparameters, and early stopping is applied to halt training when validation loss stops improving. After training, the final model is evaluated on an independent test set using metrics like accuracy, precision, recall, F1-score, and AUC-ROC to assess its performance. Finally, the trained model is optimized for deployment using TensorFlow Lite or cloud-based inference, enabling real-time, efficient, and scalable breast cancer detection.

3.4.1 **Model Training and Testing in MLP:**

The training and testing of an MLP (Multilayer Perceptron) model for breast cancer prediction follow a structured process involving data preprocessing, model initialization, training, validation, and evaluation. First, structured numerical data (such as patient demographics and biomarker levels) is normalized and split into training, validation, and testing sets. The MLP model is constructed with Dense layers, using activation functions like ReLU for hidden layers and sigmoid or soft max for the output layer, depending on whether the classification is binary or multi-class. During training, the model learns to optimize its weights using backpropagation and an optimizer like Adam or SGD, with binary or categorical cross-entropy loss functions guiding weight adjustments. Dropout layers and L2 regularization are used to prevent overfitting, while validation data helps fine-tune hyperparameters such as learning rate and batch size. Once training is complete, the model is tested on an independent dataset, and performance metrics like accuracy, precision, recall, F1-score, and AUC-ROC are used to evaluate its generalization ability.

3.5 Evaluation Metrics

Three frequently employed evaluation metrics were used to compare the performances of the proposed classification model and the compared models: "Accuracy, Precision, AUC."

1. Accuracy: Accuracy measures the overall correctness of the predictions made by the model. It is calculated as the ratio of the number of correct predictions to the total number of predictions.

$$Accuracy = \frac{TP + TN}{FP + FN + TP + TN}$$

2. Precision: Precision measures the proportion of correctly predicted positive instances (True Positives) out of all instances predicted as positive (True Positives + False Positives). It indicates how many of the predicted positive instances are positive.

$$Precision = \frac{TP}{TP + FP}$$

3. AUC: It measures how well the model distinguish between positive and negative cases.

$$\text{AUC} = \frac{TP}{TP + FP}$$

Chapter 4
System Requirements

4 SYSTEM REQUIREMENTS

The project involved analyzing the design of few applications so as to make the application more user friendly. To do so, it was really important to keep the navigations from one screen to the other well-ordered and at the same time reducing the amount of typing the user needs to do. To enhance accessibility, the application was adapted for compatibility with Python IDE, ensuring wider accessibility and ease of use for developers.

4.2 Functional Requirements

Graphical User Interface: A graphics-based operating system interface that uses icons, menus and a mouse (to click on the icon or pull down the menus) to manage interaction with the system.

4.3 Technologies and Languages used to develop

1. Python: Python is a versatile, high-level programming language known for its simplicity and readability. It offers extensive libraries and frameworks for various applications, including web development, data analysis, artificial intelligence, and more.

2. Deep Learning: Deep Learning is a subset of machine learning focused on learning representations of data using artificial neural networks with multiple layers. It enables computers to learn from large amounts of data and make complex decisions, leading to advancements in tasks like image recognition, natural language processing, and autonomous driving.

4.3.4 Debugger and Emulator

1. Google Chrome: Google Chrome is a popular web browser renowned for its compatibility and efficiency in rendering web-based applications. It provides a seamless experience for accessing the internet and running web pages.

4.3.5 Hardware Requirements

- Computer with 1.6 GHz or faster processor
- Minimum 4 GB of RAM or more
- 2.5 GB of available hard-disk

4.3.6 Software Requirements

- Operating Systems: Windows 11
- Backend- Python 3.10

Chapter 5
Design

DESIGN

5.1 System Design

The proposed system uses a Long Short-Term Memory (LSTM) model to analyze the UAV Images of Wildfire dataset as shown in Figure 5.1. It starts by preprocessing the text, extracting features, and training the model. Cross-validation techniques evaluate the model's performance. The trained model predicts fire, smoke, fog and green area on new or unseen data. External libraries like Flask, OpenCV, NumPy, TensorFlow and scikit-learn.

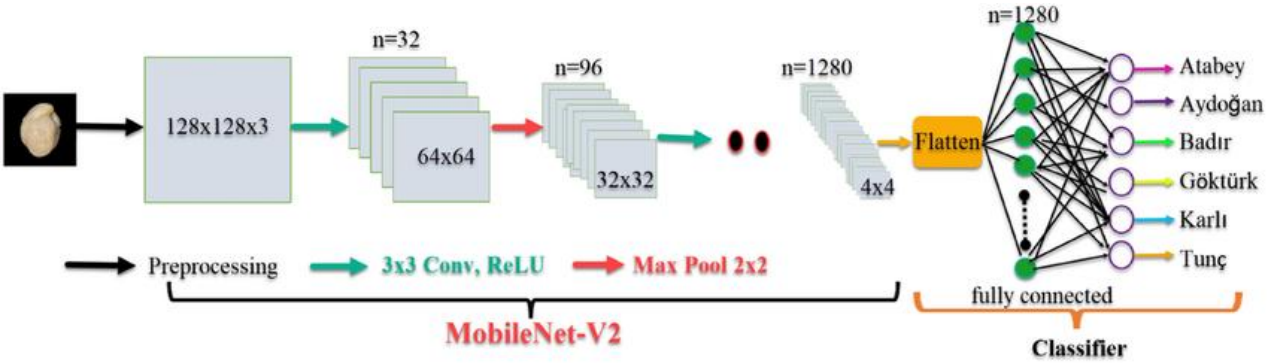


Figure 5.2: Work flow of MobileNetV2 model

The system design of MobileNetV2's working architecture follows an efficient and lightweight deep learning framework optimized for mobile and embedded applications. It begins with an input layer, where images (e.g., 128×128×3 RGB medical images) are fed into the network. The first processing stage involves a 3×3 standard convolution followed by ReLU6 activation, which captures basic low-level features such as edges and textures. The core of MobileNetV2 consists of bottleneck residual blocks, which include depth wise separable convolutions to reduce computational cost while maintaining representational power. Each block contains three stages: an expansion layer (1×1 convolution with ReLU6 activation) to increase feature depth, a depth wise 3×3 convolution for spatial filtering, and a projection layer (1×1 convolution without activation) to reduce dimensionality while preserving essential features. Down sampling occurs at specific points using stride-2 convolutions and max pooling, which progressively reduce spatial dimensions while increasing depth, enabling hierarchical feature extraction. The final feature maps are passed through a Global Average Pooling (GAP) layer, which condenses spatial information into a single 1D feature vector. This output is then fed into a fully connected layer, where a classification head processes the extracted features.

5.3 System Design of MLP model:

The system design of a Multilayer Perceptron (MLP) model follows a structured, fully connected neural network architecture that processes numerical or structured data for classification or regression tasks. The model begins with an input layer, where preprocessed data (e.g., patient biomarkers, tumor characteristics, or extracted features from medical images) is fed into the network.

Each input neuron represents a distinct feature, ensuring the model captures relevant information. The input is then passed through one or more hidden layers, each comprising multiple neurons connected via weighted edges. These hidden layers utilize Dense (fully connected) layers with activation functions like ReLU (Rectified Linear Unit) to introduce non-linearity and enhance learning capability. To improve generalization and reduce overfitting, Dropout layers are incorporated, randomly deactivating a fraction of neurons during training.

Additionally, Batch Normalization can be applied to stabilize learning by normalizing activations. The final hidden layer connects to the output layer, where a Soft max activation function is used for multi-class classification, or a Sigmoid function for binary classification, providing probability scores for each class.

The model is optimized using backpropagation and gradient descent algorithms (e.g., Adam, SGD), where the loss function (e.g., categorical or binary cross-entropy) helps in adjusting weights to minimize errors.

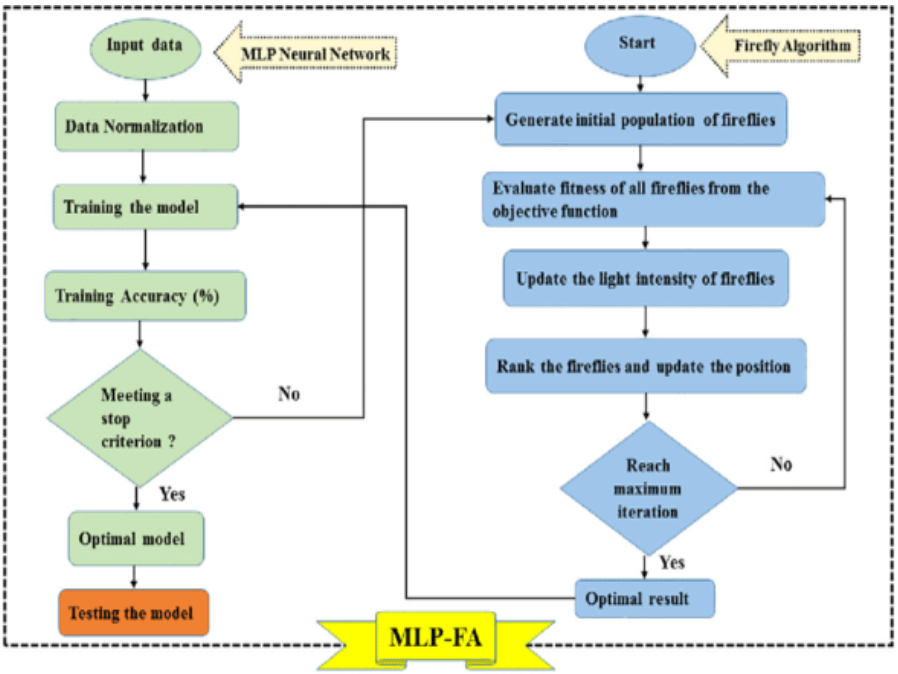


Figure 5.4: Flow Diagram

Chapter 6
Implementation

6. IMPLEMENTATION

6.1 MobileNetV2 Model:

```
# Step 1: Install Kaggle API
!pip install kaggle

# Step 2: Upload Kaggle API Key
from google.colab import files
files.upload()  # Upload kaggle.json (download it from your Kaggle account settings)

# Step 3: Move kaggle.json to the right directory
!mkdir -p ~/.kaggle
!mv kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json

# Step 4: Download dataset
!kaggle datasets download -d paultimothymooney/breast-histopathology-images

# Step 5: Extract dataset
import zipfile
import os

dataset_path = "breast-histopathology-images.zip"
extract_path = "./breast-histopathology-images"

with zipfile.ZipFile(dataset_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)

print(f"Dataset extracted to: {extract_path}")



import os
import random
import glob
import tensorflow as tf
```

```
# Constants
IMG_SIZE = 96
BATCH_SIZE = 4
EPOCHS = 10  # Reduced to 10 to prevent overfitting

# Dataset path
dataset_path = "/content/breast-histopathology-images"
breast_img_paths = glob.glob(os.path.join(dataset_path, "", "*.png"), recursive=True)
print(f"Total images found: {len(breast_img_paths)}")

# Separate classes
class_0_paths = [path for path in breast_img_paths if os.path.basename(os.path.dirname(path)) ==
'0']
class_1_paths = [path for path in breast_img_paths if os.path.basename(os.path.dirname(path)) ==
'1']
print(f"Class 0 images: {len(class_0_paths)}")
print(f"Class 1 images: {len(class_1_paths)}")

# Check if there are enough images for the new split
if len(class_0_paths) < 105000 or len(class_1_paths) < 105000:
    raise ValueError("Not enough images to assign 75,000 per class for training, 5,000 for
validation, and 25,000 for testing.")

# Shuffle and split
random.shuffle(class_0_paths)
random.shuffle(class_1_paths)
train_paths = class_0_paths[:75000] + class_1_paths[:75000]  # 150,000 total training
images
random.shuffle(train_paths)
val_paths = class_0_paths[75000:80000] + class_1_paths[75000:80000]  # 10,000 total validation
images
test_paths = class_0_paths[80000:105000] + class_1_paths[80000:105000]  # 50,000 total testing
images

# Data augmentation to reduce overfitting
```

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal_and_vertical"),
    tf.keras.layers.RandomRotation(0.2),
    tf.keras.layers.RandomZoom(0.2),
])

# Process image function
def process_path(file_path):
    label = tf.where(tf.strings.regex_full_match(file_path, ".[\\/](1)[\\/]."), 1, 0)
    img = tf.io.read_file(file_path)
    img = tf.image.decode_png(img, channels=3)
    img = tf.image.resize(img, [IMG_SIZE, IMG_SIZE])
    img = img / 255.0
    return img, label

# Datasets with augmentation for training
train_ds = tf.data.Dataset.from_tensor_slices(train_paths).map(
    process_path, num_parallel_calls=tf.data.AUTOTUNE
).map(
    lambda img, label: (data_augmentation(img, training=True), label),
num_parallel_calls=tf.data.AUTOTUNE
).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

val_ds = tf.data.Dataset.from_tensor_slices(val_paths).map(
process_path, num_parallel_calls=tf.data.AUTOTUNE
).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

test_ds = tf.data.Dataset.from_tensor_slices(test_paths).map(
    process_path, num_parallel_calls=tf.data.AUTOTUNE
).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

# Model with dropout to reduce overfitting
base_model = tf.keras.applications.MobileNetV2(
    input_shape=(IMG_SIZE, IMG_SIZE, 3), include_top=False, weights='imagenet')
base_model.trainable = True
for layer in base_model.layers[:100]:
```

```
    layer.trainable = False

model = tf.keras.Sequential([
    base_model,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dropout(0.3),  # Added dropout to prevent overfitting
    tf.keras.layers.Dense(1, activation='sigmoid')
])

# Learning rate schedule
initial_learning_rate = 0.0001  # Lowered to stabilize training
lr_schedule = tf.keras.optimizers.schedules.ExponentialDecay(
    initial_learning_rate, decay_steps=10000, decay_rate=0.9, staircase=True)

# Compile
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=lr_schedule),
    loss='binary_crossentropy',
    metrics=['accuracy', tf.keras.metrics.AUC()]  # Added AUC to monitor performance
)

# Train
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS
)

# Evaluate on test set
test_loss, test_accuracy, test_auc = model.evaluate(test_ds)
print(f"Test Loss: {test_loss}")
print(f"Test Accuracy: {test_accuracy}")
print(f"Test AUC: {test_auc}")

import tensorflow as tf
import matplotlib.pyplot as plt
```

```
import os

# Define the image size (assuming the model was trained with 96x96 images)
IMG_SIZE = 96

# Load the trained model from Google Drive
model_path = '/content/drive/MyDrive/mavyan.h5'
try:
    model = tf.keras.models.load_model(model_path)
    print("Model loaded successfully.")
except Exception as e:
    print(f"Error loading model: {e}")
    raise

# Function to classify and display a single image
def classify_and_display_image(image_path):
    try:
        # Load image for display
        img = tf.io.read_file(image_path)
        img = tf.image.decode_image(img, channels=3)  # Supports multiple image formats
        img_display = img.numpy()  # Convert to numpy array for display

        # Preprocess image for prediction (resize and normalize)
        img = tf.image.resize(img, [IMG_SIZE, IMG_SIZE])
        img = img / 255.0  # Normalize to [0, 1]
        img = tf.expand_dims(img, 0)  # Add batch dimension

        # Make prediction
        prob = model.predict(img)[0][0]  # Get probability from sigmoid output
        prediction = "Cancer" if prob > 0.5 else "No Cancer"
confidence = prob if prob > 0.5 else 1 - prob  # Confidence in the predicted class

        # Display the image with prediction
        plt.figure(figsize=(5, 5))
        plt.imshow(img_display)
        plt.title(f"Prediction: {prediction}\nConfidence: {confidence*100:.2f}%")
```

```
        plt.axis('off')
        plt.show()

        # Print the result to console
        print(f"Prediction: {prediction}, Confidence: {confidence*100:.2f}%")
    except Exception as e:
        print(f"Error processing image: {e}")

# Interactive testing loop
print("Enter the image path to classify, or type 'exit' to stop.")
while True:
    image_path = input("Image path: ")
    if image_path.lower() == 'exit':
        break
    if not os.path.exists(image_path):
        print("Error: Image path does not exist. Please try again.")
        continue
    classify_and_display_image(image_path)

print("Testing complete.")

import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, roc_curve, auc, precision_recall_curve
import seaborn as sns
import glob
import os

# Constants (adjust these if different in your training setup)
IMG_SIZE = 96  # Image size used during training (e.g., 96x96)
BATCH_SIZE = 4  # Batch size used during training

# Function to process image paths (same as used in training)
def process_path(file_path):
    # Extract label from file path (assuming '0' or '1' in directory name)
```

```
    label = tf.where(tf.strings.regex_full_match(file_path, ".[\\/](1)[\\/]."), 1, 0)
img = tf.io.read_file(file_path)
    img = tf.image.decode_png(img, channels=3)  # Assuming PNG images with 3 channels
    img = tf.image.resize(img, [IMG_SIZE, IMG_SIZE])
    img = img / 255.0  # Normalize to [0, 1]
    return img, label

# Define test data paths (adjust this based on your dataset location)
dataset_path = "/content/breast-histopathology-images"  # Example path, update as needed
breast_img_paths = glob.glob(os.path.join(dataset_path, "", "*.png"), recursive=True)
class_0_paths = [path for path in breast_img_paths if os.path.basename(os.path.dirname(path)) ==
'0']
class_1_paths = [path for path in breast_img_paths if os.path.basename(os.path.dirname(path)) ==
'1']
test_paths = class_0_paths[17500:21500] + class_1_paths[17500:21500]  # Example split, adjust
as needed

# Create test dataset
test_ds = tf.data.Dataset.from_tensor_slices(test_paths).map(
    process_path, num_parallel_calls=tf.data.AUTOTUNE
).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

# Load the trained model
model = tf.keras.models.load_model('/content/mavyan.h5')

# Make predictions on the test dataset
test_predictions = model.predict(test_ds)
pred_probs = test_predictions.flatten()  # Predicted probabilities
pred_labels = (pred_probs > 0.5).astype(int)  # Predicted labels (threshold = 0.5)

# Extract true labels from the test dataset
true_labels = []
for _, labels in test_ds:
    true_labels.extend(labels.numpy())
true_labels = np.array(true_labels)
```

```
# --- Plot 1: Confusion Matrix ---
cm = confusion_matrix(true_labels, pred_labels)
plt.figure(figsize=(6, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['No Cancer', 'Cancer'], yticklabels=['No Cancer', 'Cancer'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()

# --- Plot 2: ROC Curve ---
fpr, tpr, _ = roc_curve(true_labels, pred_probs)
roc_auc = auc(fpr, tpr)
plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic')
plt.legend(loc="lower right")
plt.show()

# --- Plot 3: Precision-Recall Curve ---
precision, recall, _ = precision_recall_curve(true_labels, pred_probs)
pr_auc = auc(recall, precision)
print(f"Precision-Recall AUC: {pr_auc:.4f}")

plt.figure()
plt.plot(recall, precision, color='blue', lw=2, label=f'PR curve (area = {pr_auc:.2f})')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve')
plt.legend(loc="lower left")
plt.show()
```

```
# Evaluate the model on the test set
test_loss, test_accuracy, test_auc = model.evaluate(test_ds)
print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")
print(f"Test AUC: {test_auc:.4f}")
```

6.2 MLP Model:

```
# Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score, roc_auc_score
import tensorflow as tf
from keras.models import Sequential
from keras.layers import Dense, Dropout, BatchNormalization
from keras.callbacks import EarlyStopping

# Load your dataset (replace with your file path)
df = pd.read_csv("/content/dataR2.csv")

# Rename the target column if needed (adjust 'Classification' to your label column name)
df = df.rename(columns={'Classification': 'Label'})

# Handle missing values
df = df.dropna()
print("Dataset Summary:")
print(df.describe().T)
print("\nMissing Values:")
print(df.isnull().sum())

# Visualize label distribution
sns.countplot(x=df["Label"])
plt.title("Label Distribution")
plt.show()
```

```
# Encode categorical labels (e.g., 0 for benign, 1 for malignant)
le = LabelEncoder()
df["Label"] = le.fit_transform(df["Label"])
y = df["Label"].values

# Define features (drop the target column)
X = df.drop(columns=["Label"], axis=1)

# Scale the features
scaler = MinMaxScaler()
X = pd.DataFrame(scaler.fit_transform(X), columns=X.columns)

# Split data with reduced test size (20% test, 80% train)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(f"Training set size: {X_train.shape[0]} samples")
print(f"Test set size: {X_test.shape[0]} samples")

# Build the neural network model
model = Sequential([
    Dense(64, input_dim=X_train.shape[1], activation='relu'),
    BatchNormalization(),
    Dropout(0.3),
    Dense(16, activation='relu'),
    BatchNormalization(),
    Dropout(0.3),
    Dense(8, activation='relu'),
    Dense(1, activation='sigmoid')
])

# Compile the model with accuracy and AUC metrics
model.compile(loss='binary_crossentropy', optimizer='adam',
```

```
            metrics=['accuracy', tf.keras.metrics.AUC(name='auc')])
print("\nModel Summary:")
print(model.summary())

# Define early stopping to prevent overfitting
early_stopping = EarlyStopping(monitor='val_auc', patience=20, mode='max',
restore_best_weights=True)

# Train the model
history = model.fit(X_train, y_train, epochs=100, batch_size=32, verbose=1,
                    validation_data=(X_test, y_test), callbacks=[early_stopping])

# Plot training and validation loss
plt.plot(history.history['loss'], 'y', label='Training Loss')
plt.plot(history.history['val_loss'], 'r', label='Validation Loss')
plt.title('Training and Validation Loss')
plt.plot(history.history['val_loss'], 'r', label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()

# Plot training and validation accuracy
plt.plot(history.history['accuracy'], 'y', label='Training Accuracy')
plt.plot(history.history['val_accuracy'], 'r', label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

```
# Predict probabilities on the test set
y_pred_probs = model.predict(X_test)

# Optimize threshold for best accuracy
thresholds = np.arange(0.1, 0.9, 0.1)
accuracies = []
for thresh in thresholds:
    y_pred = (y_pred_probs > thresh).astype(int)
    acc = accuracy_score(y_test, y_pred)
    accuracies.append(acc)

best_thresh = thresholds[np.argmax(accuracies)]
best_accuracy = max(accuracies)
print(f"\nBest threshold: {best_thresh:.1f}, with accuracy: {best_accuracy * 100:.2f}%")

# Final predictions using the best threshold
y_pred = (y_pred_probs > best_thresh).astype(int)

# Evaluate with confusion matrix
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d')
plt.title('Confusion Matrix with Optimized Threshold')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# Print final metrics
accuracy = accuracy_score(y_test, y_pred)
auc_score = roc_auc_score(y_test, y_pred_probs)
print(f"Test Accuracy with Optimized Threshold: {accuracy * 100:.2f}%")
print(f"Test AUC: {auc_score:.4f}")
```

```
# Function to predict new data (optional)
def predict_breast_cancer(new_data):
    new_data = pd.DataFrame([new_data], columns=X.columns)
    new_data = scaler.transform(new_data)
    prediction = model.predict(new_data)
    return "Cancerous (Malignant)" if prediction > best_thresh else "Non-
Cancerous (Benign)"
```

Chapter 7

Results

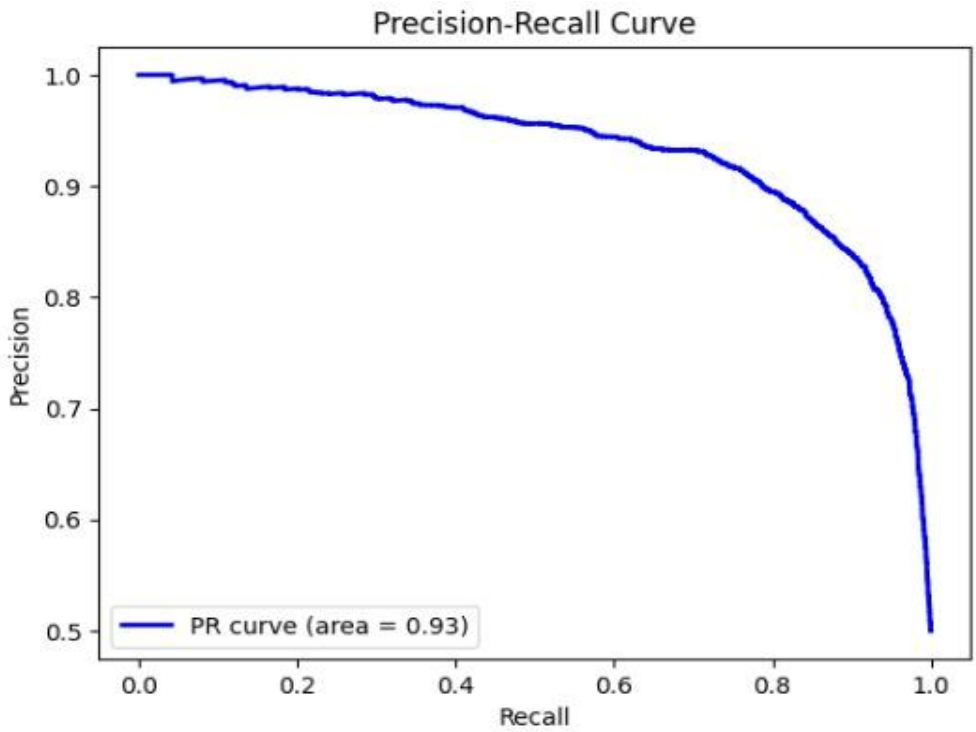
7. RESULTS

The model is compared with the existing model and observed that the proposed model achieves more accuracy, AUC and ROC values as you can see in table 7.1. The proposed method achieves an accuracy of 85% and AUC of 97%, ROC of 93%.

Table 7.1 Results of MobileNetV2 vs MLP

Model	Accuracy	AUC	ROC
CNN	83.7%	88.6%	87.23%
CRNN	85.6%	93.5%	91.9%
MLP	87.5%	97.92%	94.1%

In the confusion matrix of Fig 7.1, the proposed method shows a better ability to classify all classes and in Fig 7.2 you can find the existing method confusion matrix.




```
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 18s 7ms/step - accuracy: 0.8733 - auc: 0.4714 - loss: 0.2977
Test Loss: 0.3465
Test Accuracy: 0.8560
Test AUC: 0.9350
```

Figure 7.2 AUC Curve of MobileNetV2 Model

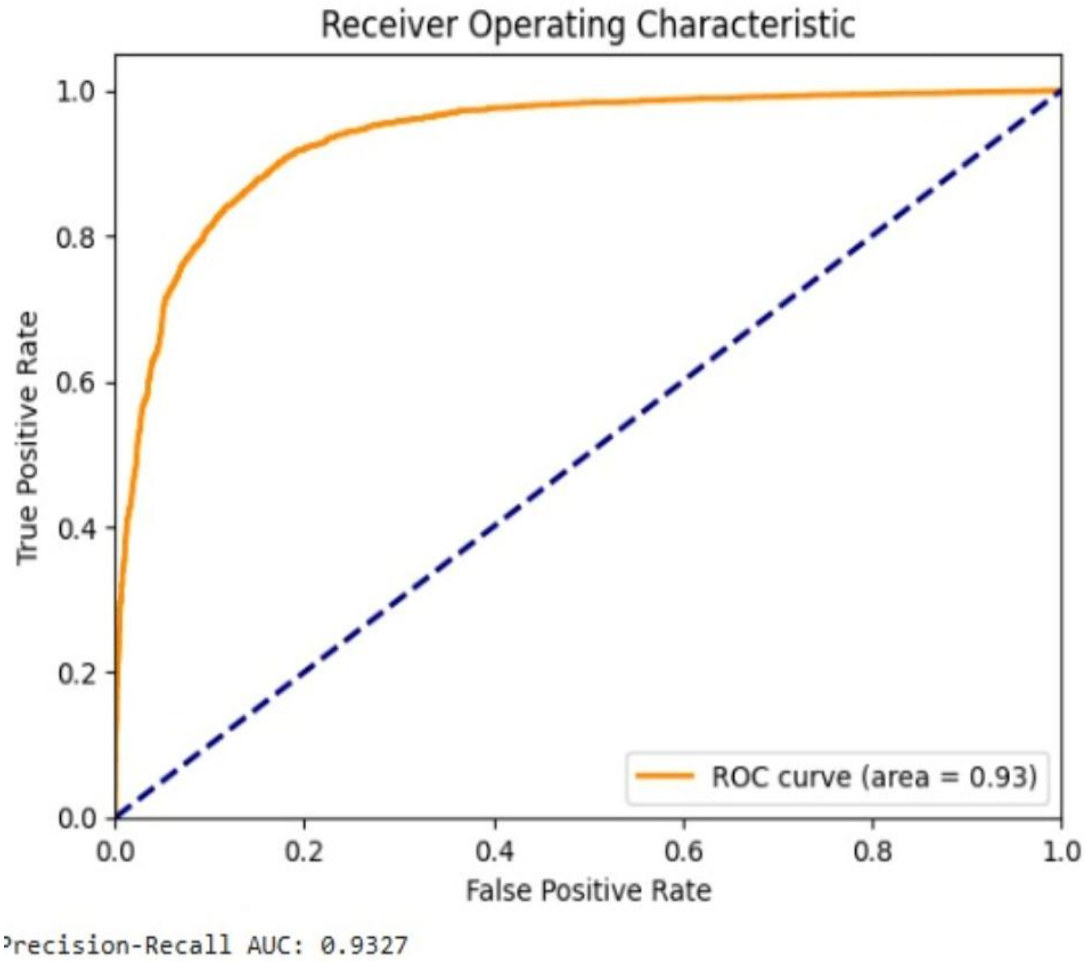


Figure 7.3 ROC Curve of MobileNetV2 Model

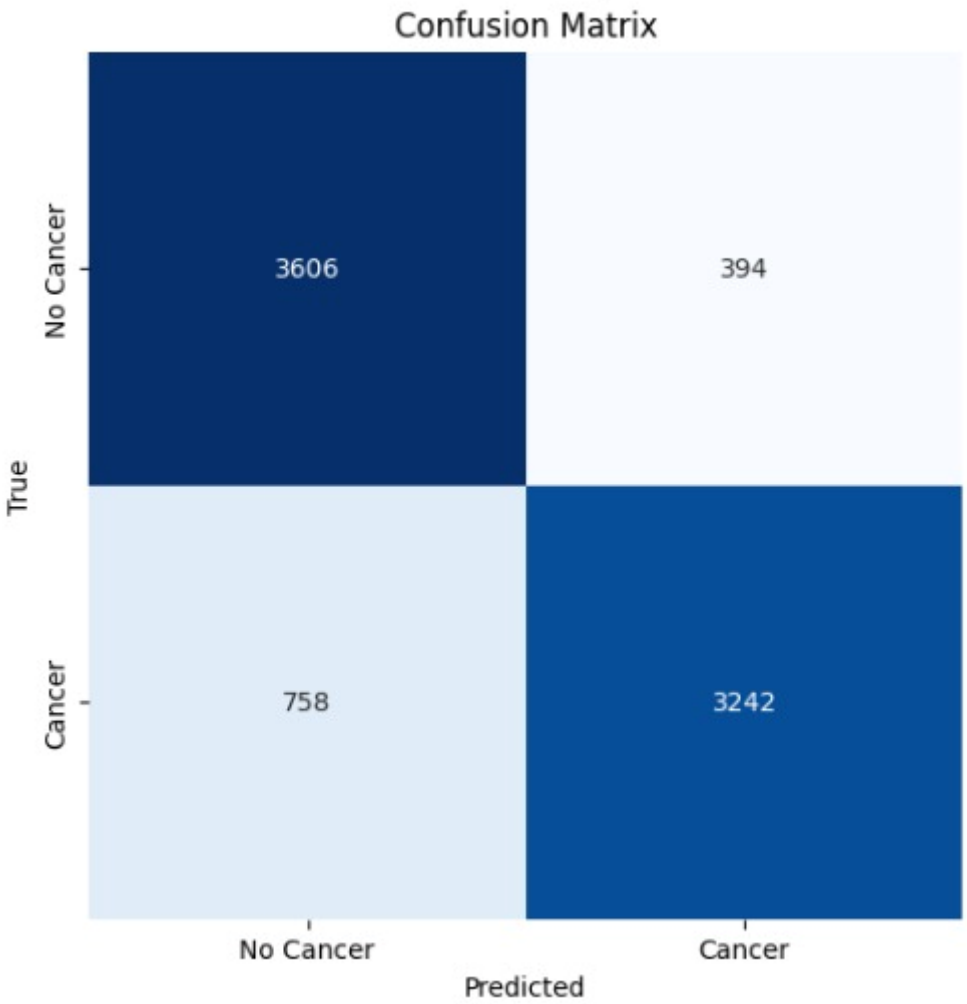


Fig 7.4 Confusion Matrix of MobileNetV2 model

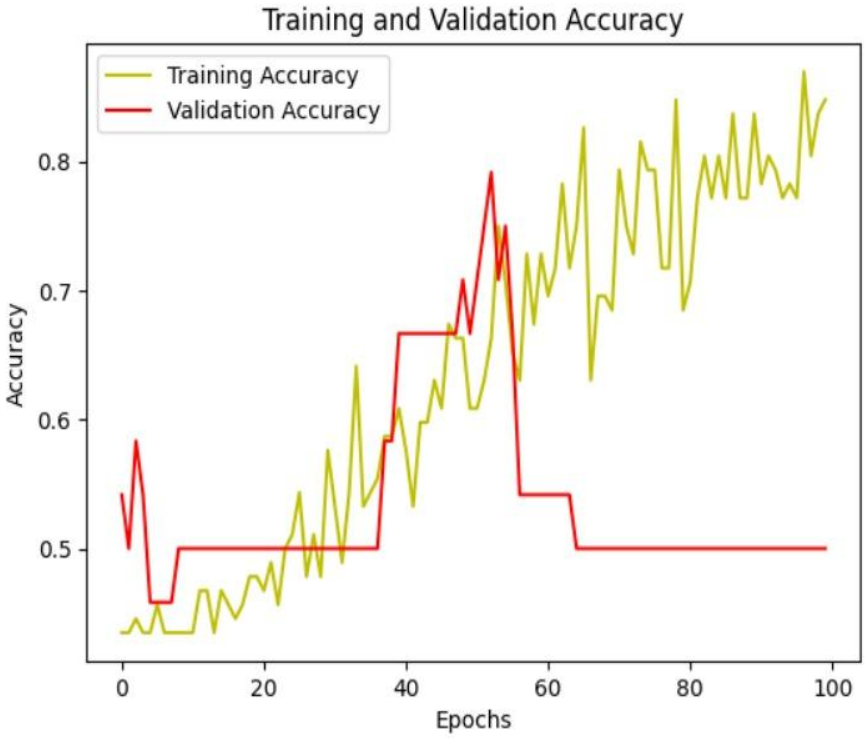


Figure 7.5 AUC Curve of MLP Model

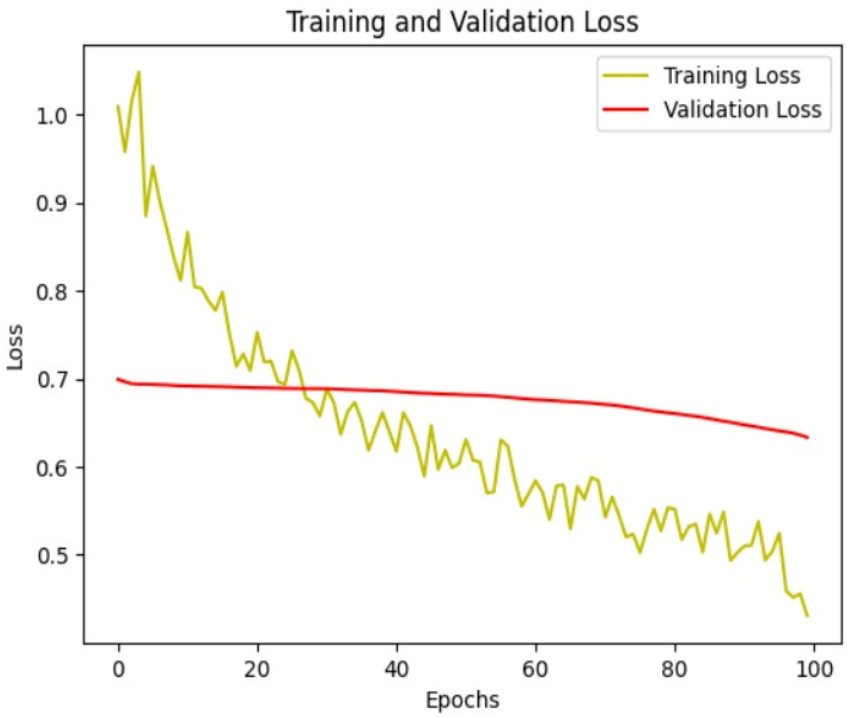
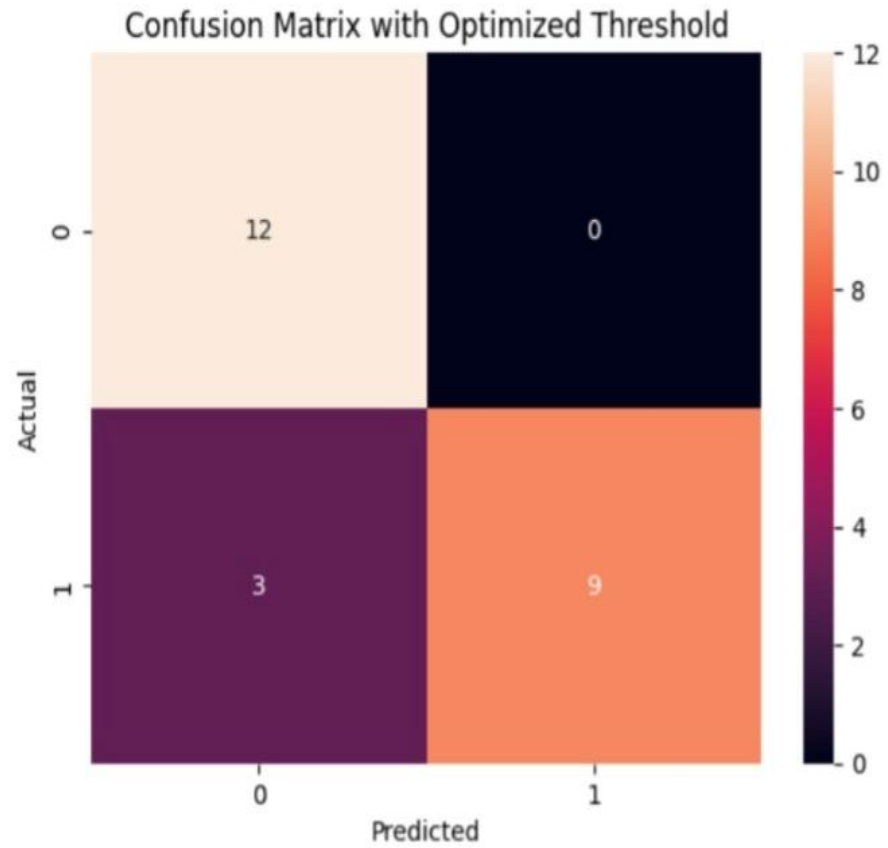


Figure 7.6 ROC Curve of MLP Model



Test Accuracy with Optimized Threshold: 87.50%
Test AUC: 0.9792

Fig 7.7 Confusion Matrix of MLP Model

Chapter 8

Social Impact

8. SOCIAL IMPACT

The social impact of breast cancer detection using MobileNetV2 and MLP models extends beyond just improving diagnosis—it fundamentally transforms the accessibility, affordability, and efficiency of healthcare services. MobileNetV2, being a lightweight deep learning model, enables real-time breast cancer detection on low-power devices such as smartphones and edge AI systems, allowing for mass screening in rural and underserved regions where access to expert radiologists is limited.

This mobile-friendly AI-powered screening solution can assist community healthcare workers, reducing dependency on specialized infrastructure and enabling quicker decision-making. On the other hand, the MLP model, designed to process structured patient data, plays a crucial role in integrating diverse medical parameters such as genetic markers, hormone levels, and clinical history, offering a more comprehensive risk assessment. Combining imaging-based MobileNetV2 predictions with MLP-driven patient history analysis can significantly enhance diagnostic accuracy, reducing false positives and negatives that often lead to unnecessary treatments or missed detections. Additionally, the deployment of these models in telemedicine platforms and cloud-based AI-assisted diagnostics fosters global healthcare inclusivity, ensuring that even patients in resource-constrained settings receive expert-level assessments. AI-driven early detection also alleviates the workload of radiologists by automating preliminary screenings, allowing specialists to focus on complex cases and improving overall hospital efficiency. Economically, AI-powered detection models lower healthcare costs by reducing the need for expensive biopsy procedures, unnecessary imaging tests, and prolonged hospital visits.

Psychologically and socially, early and accurate diagnosis leads to timely interventions, increasing survival rates and improving patient outcomes, which in turn reduces the emotional and financial burden on families. Furthermore, AI-powered awareness campaigns and mobile health applications can educate women about self-examinations, early symptoms, and risk factors, encouraging proactive health-seeking behaviour and leading to better public health outcomes. By democratizing cancer detection and making advanced healthcare solutions widely accessible, MobileNetV2 and MLP models have the potential to revolutionize global breast cancer care, saving countless lives while bridging healthcare disparities.

Chapter 9

Conclusion & Future Work

9. CONCLUSION & FUTURE WORK

The implementation of MobileNetV2 and MLP models for breast cancer detection has demonstrated significant potential in improving the accuracy, accessibility, and efficiency of diagnostic procedures. MobileNetV2's lightweight architecture enables real-time, resource-efficient image classification, making it suitable for deployment on mobile devices and edge computing platforms, thereby expanding access to early detection in underserved areas. The MLP model, with its structured data analysis capability, enhances decision-making by integrating clinical parameters, improving the reliability of predictions beyond image-based analysis. Together, these models offer a cost-effective, AI-driven diagnostic approach, reducing reliance on manual interpretation and enabling faster, more consistent medical assessments. This integration of AI into breast cancer detection has the potential to revolutionize healthcare, offering improved patient outcomes through early intervention and timely treatment.

- **Exploring Advanced Techniques:** Further research could investigate the integration of advanced techniques such as attention mechanisms or ensemble learning to enhance the model's performance. These techniques could improve the model's ability to discern relevant features in the imagery datasets, leading to more accurate wildfire detection.
- **Mobile Application Development:** Developing a mobile application could extend the project's utility by providing real-time wildfire alerts to users. By integrating the trained CRNN model into the app, individuals in wildfire-prone areas could receive timely notifications, enabling them to take necessary precautions and evacuate if needed.
- **Multi-Modal Learning for Enhanced Accuracy** – Combining MobileNetV2's image-based analysis with MLP's structured clinical data processing will improve diagnostic precision, reducing false positives and negatives by leveraging both visual and non-visual patient information.

BIBLIOGRAPHY

[1] Wilkinson Louise, Gathani Toral. Understanding breast cancer as a global health concern. Br J Radiol 2022;95(1130):20211033.

[2] Siegel Rebecca L, Miller Kimberly D. Nikita sandeep wagle, and ahmedin jemal. Cancer statistics, 2023. CA: A Cancer Journal for Clini- cians 2023;73(1):17–48. https://doi.org/10.3322/caac.21763.https://acsjournals.onlinelibrary.wiley.com/doi/abs/10.3322/caac.21763. URL.

[3] Pfei er Ruth M, Webb-Vargas Yenny, Wheeler William, Mitchell H Gail. Proportion of us trends in breast cancer incidence attributable to long-term changes in risk factor distributions. Cancer Epidemiol Biomarkers Prev 2018;27(10):1214–22.

[4] Torres-Galvan Juan Carlos, Guevara Edgar, Javier Gonzalez Francisco. Comparison of deep learning architectures for pre-screening of breast cancer thermo- grams. In: 2019 photonics north (PN). IEEE; 2019. pages 1{2.

[5] Debelee Taye Girma, Schwenker Friedhelm, Ibenthal Achim, Yohannes Dereje. Survey of deep learning in breast cancer image analysis. Evolving Systems 2020;11 (1):143–63.

[6] Yu Keping, Tan Liang, Lin Long, Cheng Xiaofan, Zhang Yi, Sato Takuro. Deeplearning-empowered breast cancer auxiliary diagnosis for 5gb remote e-health. IEEE Wireless Commun 2021;28(3):54–61.

[7] Zhou Li-Qiang, Wu Xing-Long, Huang Shu-Yan, Wu Ge-Ge, Ye Hua-Rong, Qi Wei, Bao Ling-Yun, Deng You-Bin, Li Xing-Rui, Cui Xin-Wu, et al. Lymph node metastasis prediction from primary breast cancer us images using deep learning. Radiology 2020;294(1):19–28.

[8] Zheng Jing, Lin Denan, Gao Zhongjun, Wang Shuang, He Mingjie, Fan Jipeng. Deep learning assisted e cient adaboost algorithm for breast cancer detection and early diagnosis. IEEE Access 2020;8:96946–54.

[9] Bai Jun, Posner Russell, Wang Tianyu, Yang Cli ord, Nabavi Sheida. Ap plying deep learning in digital breast tomosynthesis for automatic breast cancer detection: a review. Med Image Anal 2021;71:102049.

[10] Yala Adam, Lehman Constance, Schuster Tal, Portnoi Tally, Barzilay Regina. A deep learning mammography-based model for improved breast cancer risk prediction. Radiology 2019;292(1):60–6.

[11] Naik Nikhil, Ali Madani, Esteva Andre, Keskar Nitish Shirish, Press Michael F, Ruderman Daniel, Agus David B, Socher Richard. Deep learning-enabled breast cancer hormonal receptor status determination from base-level h&e stains. Nat Commun 2020;11(1):1–8.

[12] Khuriwal Naresh, Mishra Nidhi. Breast cancer diagnosis using deep learning algorithm. In: 2018 international conference on advances in computing, communication control and networking (ICACCCN). IEEE; 2018. p. 98–103.

[13] Street W Nick, Wolberg William H, Mangasarian Olvi L. Nuclear feature extraction for breast

tumor diagnosis. In: Biomedical image processing and biomedical visualization. vol. 1905. SPIE; 1993. p. 861–70.

[14] Crisostomo Joana, Matafome Paulo, Santos-Silva Daniela, Gomes Ana L, Gomes Manuel, cio Miguel Patr, Letra Liliana, Sarmento-Ribeiro Ana B, Santos Lelita, Seica Raquel. Hyperresistinemia and metabolic dysregulation: a risky crosstalk in obese breast cancer. Endocrine 2016;53(2):433–42.

[15] Lake JK, Power C, Cole TJ. Women's reproductive health: the role of body mass index in early and adult life. Int J Obes 1997;21(6):432–8.